

Name:

Rollno:

CS345: Design and Analysis of Algorithms (Quiz 2)

8th Sept 2026

Total Number of Pages: 6

Time: 1hr

Total Points 40

Instructions

1. All questions are compulsory.
2. Answer all the questions in the space provided in the question paper booklet.
3. Use the space provided in the paper for rough work.
4. The symbols or notations mean as usual unless stated.
5. You may cite and use algorithms and their complexity as done in the class.
6. Cheating or resorting to any unfair means will be severely penalized.
7. Superfluous and irrelevant writing will result in negative marking.
8. Use blue/black ink and not pencils. Do not use red pens.

Question	Points	Score
1	3	
2	3	
3	3	
4	3	
5	3	
6	3	
7	3	
8	3	
9	3	
10	3	
11	10	
Total:	40	

Helpful hints

1. It is advisable to solve a problem first before writing down the solution.
 2. The questions are *not* arranged according to increasing order of difficulty.
-

Name:

Rollno:

Question 1. (3 points) Consider the recurrence relation

$$T(n) = 4T(n/2) + n^2 \log n.$$

Determine the asymptotic time complexity of $T(n)$.

Answer: $\Theta(n^2 \log^2 n)$

Solution: Recursion tree: level i has 4^i subproblems of size $n/2^i$, so its cost is $4^i \cdot (n/2^i)^2 \log(n/2^i) = n^2(\log n - i)$. Summing over $i = 0, \dots, \log n$ gives $n^2 \sum_{k=0}^{\log n} k = \Theta(n^2 \log^2 n)$. (Equivalently $n^{\log_2 4} = n^2$ and $f(n) = \Theta(n^2 \log n)$, which is Case 2 of the Master Theorem with one extra log factor.)

Question 2. (3 points) In the divide and conquer algorithm for the closest pair of points, let $\delta = \min(\delta_L, \delta_R)$ after the two recursive calls. In the combine step, for a point p of the left strip we examine only those points q of the right strip that satisfy $y(p) - \delta \leq y(q) \leq y(p) + \delta$. Why is the number of such points q bounded by a constant?

- A. The right strip contains only $O(1)$ points in total.
- B. These points lie inside a $\delta \times 2\delta$ rectangle contained in the right half, and every two points of the right half are at distance at least δ .**
- C. The right strip is stored sorted by y -coordinate, so a binary search returns only $O(1)$ of them.
- D. δ is the maximum the distance between two consecutive points in the y -sorted order of the strip.

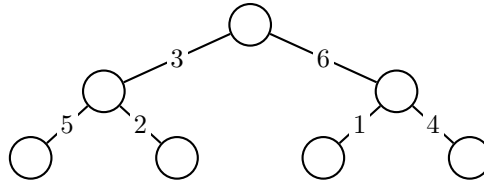
Solution: Such a q lies within horizontal distance δ of the dividing line and within vertical distance δ of p , i.e. inside a $\delta \times 2\delta$ rectangle lying entirely in the right half. Any two points of the right half are at distance at least $\delta_R \geq \delta$, and by the packing argument (a $\delta \times \delta$ square holds at most 4 such points) such a rectangle contains at most a constant number of points. Hence the combine step costs $O(1)$ per point of the strip. Note that (C) is wrong as stated: sorting tells us *how to find* the candidates quickly, it does not bound *how many* there are.

Question 3. (3 points) A height balanced BST represents a sequence $e_1, e_2, \dots, e_n$, keyed by position. Every node v stores a field **value** and a field **incr**, where adding x to **incr**(v) means “add x to every element of the subtree rooted at v ”. Which of the following statements are correct?

- I.** $\text{Add}(T, i, j, x)$ modifies the **incr** field of $O(\log n)$ nodes.
 - II.** $\text{Add}(T, i, j, x)$ modifies the **incr** field of $O(j - i)$ nodes.
 - III.** The element stored at a node u equals **value**(u) plus the sum of **incr** over all ancestors of u , u itself included.
 - IV.** The element stored at a node u equals **value**(u) + **incr**(u).
- A. I and III**
 - B. I and IV
 - C. II and III
 - D. II and IV

Solution: The interval $[i, j]$ is covered by $O(\log n)$ canonical subtrees hanging off the two root-to- i and root-to- j paths, and only the roots of these subtrees get their **incr** field incremented, so I holds and II does not. An **incr** value stored at an ancestor is a *pending* addition that applies to the whole subtree, so all of them must be accumulated along the root-to- u path; hence III, and not IV.

Question 4. (3 points) The circuit below is a complete binary tree of depth 2; the number on an edge is its delay. Delays may only be *increased*. What is the minimum total delay enhancement needed to make the delays of all four root-to-leaf paths equal?



- A. 7
- B. 8
- C. 9
- D. 10

Solution: Work bottom-up, as in the algorithm done in class. At the left child the two leaf edges are 5 and 2, so raise 2 to 5 at a cost of 3; the subtree now has height 5. At the right child the leaf edges are 1 and 4, so raise 1 to 4 at a cost of 3; that subtree has height 4. At the root the two path delays are $3 + 5 = 8$ and $6 + 4 = 10$, so raise the edge of delay 3 by 2. Total = $3 + 3 + 2 = 8$, and all four root-to-leaf delays become 10. (An exhaustive search over all ways of increasing the six edge delays confirms 8 is optimal.)

Question 5. (3 points) Which of the following multisets of codeword lengths cannot be the multiset of codeword lengths of a binary prefix code for an alphabet of 6 symbols?

- A. $\{1, 2, 3, 4, 5, 5\}$
- B. $\{2, 2, 2, 3, 4, 4\}$
- C. $\{1, 3, 3, 3, 4, 4\}$
- D. $\{1, 2, 2, 3, 4, 4\}$

Solution: A prefix code corresponds to a binary tree whose leaves are the symbols, and the tree of an optimal code is a *full* binary tree; both together are equivalent to $\sum_i 2^{-\ell_i} = 1$. The first three multisets give the sum exactly 1 (and each is optimal, for instance for frequencies proportional to $2^{-\ell_i}$), whereas $\{1, 2, 2, 3, 4, 4\}$ gives $\frac{1}{2} + \frac{1}{4} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{16} = 1.25 > 1$, so it is not even a prefix code.

Question 6. (3 points) Let τ be a topological ordering of a DAG G , and suppose we wish to compute a function f on the vertices, where $f(y)$ is defined in terms of the values $\{f(z) : (y, z) \in E\}$ of the *out*-neighbours of y only. In which order should the vertices be processed so that a single non-recursive pass computes every value correctly?

Name:

Rollno:

- A. Increasing order of τ .
- B. Decreasing order of τ .**
- C. Increasing order of in-degree.
- D. Decreasing order of out-degree.

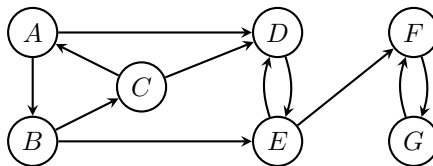
Solution: Every out-neighbour z of y satisfies $\tau(z) > \tau(y)$, so $f(z)$ must already be available when $f(y)$ is computed. Processing the vertices in decreasing order of τ (the sinks first) guarantees exactly this. Increasing order of τ is the right order when $f(y)$ is defined from the in-neighbours instead.

Question 7. (3 points) Let (u, v) be an edge of a directed graph G , and suppose a DFS of G gives $F[u] < F[v]$. Then (u, v) must be

- A. a tree edge
- B. a forward edge
- C. a backward edge**
- D. a cross edge

Solution: If (u, v) is a tree or a forward edge then v is a descendant of u , so the interval $[D[v], F[v]]$ is nested inside $[D[u], F[u]]$ and $F[v] < F[u]$. If it is a cross edge then v was already finished when the edge was explored, so again $F[v] < F[u]$. Hence $F[u] < F[v]$ forces v to be an ancestor of u , i.e. (u, v) is a backward edge. (This is exactly the test used to detect a cycle in a directed graph.)

Question 8. (3 points) Consider the directed graph shown below.



How many strongly connected components does this graph have?

- A. 2
- B. 3**
- C. 4
- D. 5

Solution: The components are $\{A, B, C\}$ (the cycle $A \rightarrow B \rightarrow C \rightarrow A$), $\{D, E\}$ and $\{F, G\}$. The edges $A \rightarrow D$, $C \rightarrow D$, $B \rightarrow E$ and $E \rightarrow F$ are inter-component edges: no vertex of $\{D, E, F, G\}$ has an edge back into $\{A, B, C\}$, and neither F nor G has an edge back into $\{D, E\}$.

Question 9. (3 points) The algorithm discussed in class for computing strongly connected components first performs a DFS of G to compute the finish times, and then repeatedly picks the unvisited vertex with the largest finish time and extracts its component. Number the components $1, 2, 3, \dots$ in the order in which they are extracted. This sequence of components is

Name:

Rollno:

- A. a topological ordering of the SCC graph of G .
- B. the reverse of a topological ordering of the SCC graph of G .
- C. an ordering by non-increasing size of the components.
- D. arbitrary; no ordering property is guaranteed.

Solution: If there is an edge from a component C to a component C' then $F[C] > F[C']$, where $F[X]$ is the largest finish time in X . The algorithm always picks the largest remaining finish time, so C is always extracted before C' ; that is precisely the statement that the components come out in a topological ordering of the (acyclic) SCC graph. In particular the first component extracted is a source of the SCC graph.

Question 10. (3 points) Let $L(i, j)$ denote the length of a longest common subsequence of $X[1..i]$ and $Y[1..j]$. The number of calls made by the naive recursive algorithm satisfies $T(n, m) = T(n-1, m) + T(n, m-1)$ with $T(n, 0) = T(0, m) = 1$. Which of the following is correct?

- A. $T(n, n) = \Theta(2^n)$, and the number of distinct subproblems is $\Theta(2^n)$.
- B. $T(n, n) = \binom{2n}{n}$, and the number of distinct subproblems is $(n+1)^2$.
- C. $T(n, n) = \Theta(n^2)$, and the number of distinct subproblems is $(n+1)^2$.
- D. $T(n, n) = \binom{2n}{n}$, and the number of distinct subproblems is $\binom{2n}{n}$.

Solution: The recurrence with those boundary values gives $T(n, m) = \binom{n+m}{n}$, hence $T(n, n) = \binom{2n}{n} = \Theta(4^n / \sqrt{n})$, which is exponential. On the other hand a subproblem is completely described by the pair (i, j) with $0 \leq i, j \leq n$, so there are only $(n+1)^2$ distinct subproblems. This gap between the number of calls and the number of distinct subproblems is exactly the overlap that dynamic programming removes, bringing the cost down to $\Theta(n^2)$.

Name:

Rollno:

Question 11. (10 points) Let $S[1..n]$ be a string. A subsequence of S is called a *palindromic subsequence* if it reads the same from left to right as from right to left (for example ACBCA and BB are palindromic subsequences of ABCBCBA). Design an $O(n^2)$ time algorithm that computes the *length* of a longest palindromic subsequence of S . State the recursive formulation together with its base cases, justify it briefly, and mention the order in which the subproblems are computed as well as the time and space complexity of your algorithm.

Solution: Subproblem. For $1 \leq i \leq j \leq n$, let $P[i][j]$ denote the length of a longest palindromic subsequence of the substring $S[i..j]$. We also define $P[i][i-1] = 0$ (empty string). The required answer is $P[1][n]$.

Recursive formulation. $P[i][i] = 1$ for every i , and for $i < j$

$$P[i][j] = \begin{cases} 2 + P[i+1][j-1], & \text{if } S[i] = S[j], \\ \max(P[i+1][j], P[i][j-1]), & \text{if } S[i] \neq S[j]. \end{cases}$$

Justification. *Case $S[i] \neq S[j]$.* The first and the last character of a palindrome are equal, so no palindromic subsequence of $S[i..j]$ can use both $S[i]$ and $S[j]$. Hence every palindromic subsequence of $S[i..j]$ is a palindromic subsequence of $S[i+1..j]$ or of $S[i..j-1]$, and conversely each of these is one of $S[i..j]$. Therefore $P[i][j]$ is the larger of the two.

Case $S[i] = S[j]$ (with $i < j$). Placing $S[i]$ and $S[j]$ around an optimal palindromic subsequence of $S[i+1..j-1]$ shows $P[i][j] \geq 2 + P[i+1][j-1]$. For the reverse inequality, let Q be any longest palindromic subsequence of $S[i..j]$, occupying indices $a_1 < a_2 < \dots < a_k$. Since $S[i]S[j]$ is itself a palindrome, $k \geq 2$. Replacing a_1 by i and a_k by j is legal, because $i \leq a_1 < a_2$ and $a_{k-1} < a_k \leq j$; as $S[i] = S[j]$, the new sequence is still a palindromic subsequence of the same length. So some optimal solution uses both $S[i]$ and $S[j]$, and removing these two characters leaves a palindromic subsequence of $S[i+1..j-1]$ of length $P[i][j] - 2$, which cannot exceed $P[i+1][j-1]$. Hence $P[i][j] = 2 + P[i+1][j-1]$.

Algorithm (bottom-up). $P[i][j]$ depends only on entries of strictly smaller length $j - i + 1$, so the subproblems are computed in increasing order of the length $\ell = j - i + 1$.

```
for i = 1 to n: P[i][i] = 1; P[i][i-1] = 0
for l = 2 to n:
  for i = 1 to n-l+1:
    j = i+l-1
    if S[i] = S[j] then P[i][j] = 2 + P[i+1][j-1]
    else P[i][j] = max(P[i+1][j], P[i][j-1])
return P[1][n]
```

Complexity. There are $\Theta(n^2)$ subproblems and each is solved in $O(1)$ time from already computed entries, so the running time is $\Theta(n^2)$ and the space used is $\Theta(n^2)$.

Alternative accepted solution. The length of a longest palindromic subsequence of S equals $\text{LCS}(S, S^R)$, where S^R is the reverse of S ; invoking the $O(n^2)$ LCS algorithm done in class then gives the bound. (Full credit only with a justification of this equality.)